

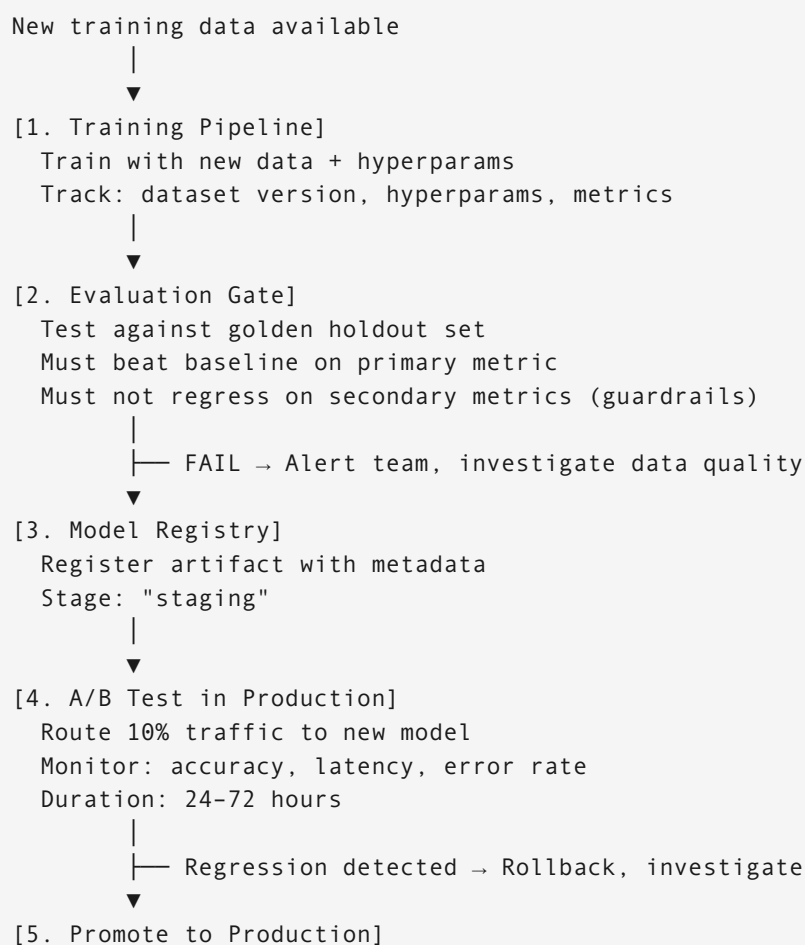
Section 11.10 of 10: MLOps/LLMOps Lifecycle & ML Platform Infrastructure

Executive Overview

MLOps/LLMOps operationalizes machine learning: turning trained models into reliable, observable, continuously improving production systems. The discipline covers model registries, CI/CD pipelines for ML, feature stores, evaluation frameworks, and hallucination detection. For LLMs, it adds LLM-specific concerns: prompt versioning, RAG evaluation, and output quality monitoring at scale.

11.10.1 Model Lifecycle & Versioning

Train → Register → Deploy Loop



```
Stage: "production"
Previous version → "archived"
Old version still available for rollback
```

Model Registry — Version Control

```
import mlflow

# Register a trained model
mlflow.register_model(
    model_uri="runs:/abc123/churn_model",
    name="churn-prediction-v2"
)

# Lifecycle stages
mlflow.transition_model_version_stage(
    name="churn-prediction-v2",
    version=5,
    stage="Production",
    archive_existing_versions=True # Auto-archive old production
)

# Required metadata for every version:
mlflow.log_params({
    "training_data_version": "2025-06-01",
    "training_samples": 2_500_000,
    "model_architecture": "xgboost-v2",
    "owner": "ml-team@company.com"
})
mlflow.log_metrics({
    "auc_roc": 0.87,
    "precision@10": 0.72,
    "inference_p99_ms": 12
})
```

Immutability Principle

Rule: Never modify a deployed model artifact.

```
Version 5 in production:
Artifact hash: sha256:abc123... (immutable)
Any change = increment version number
Version 4 remains archived (instant rollback)
```

Why immutability matters:

1. Reproducibility: Re-run version 5 at any time with identical behavior
2. Audit trail: "What model served request X at time T?" → unambiguous answer
3. Safe experiments: Deploy version 6 alongside version 5; rollback in <60 seconds

Rollback procedure:

1. Detect regression (automated alert or manual)
 2. `mlflow.transition_stage("v5", "Production")`
 3. `mlflow.transition_stage("v6", "Archived")`
 4. Load balancer already points to registry → no deploy needed
- Total time: <5 minutes

11.10.2 Feature Stores & Train-Serve Skew

The Train-Serve Skew Problem

Training (offline):

```
SELECT age, churn_risk FROM users WHERE event_date <= '2025-01-15'
```

→ Model trained on features as of Jan 15

Serving (online, months later):

```
SELECT age, churn_risk FROM users WHERE user_id = 123
```

→ Returns current values (not Jan 15 values)

Problem: Feature "churn_risk" was recomputed with new logic on Feb 1.
Model sees different feature distribution than it trained on.
Prediction accuracy silently degrades.

Point-in-Time Correctness

```
# Training: retrieve features AS OF each event timestamp
training_features = feature_store.get_historical_features(
    entity_df=pd.DataFrame({
        "user_id": [1, 2, 3],
        "event_timestamp": ["2025-01-10", "2025-01-12", "2025-01-15"]
    }),
    features=["user_stats:avg_session_duration", "user_stats:days_since_login"]
)
# Returns values that existed AT event_timestamp – not current values

# Serving: retrieve latest features
serving_features = feature_store.get_online_features(
    features=["user_stats:avg_session_duration"],
    entity_rows=[{"user_id": 123}]
)
```

Feature Freshness Strategy

Feature Type	Update Frequency	Storage	Use When
Batch	Hourly / daily	Feature store (Feast, Tecton)	Historical aggregates, slow-moving signals
Streaming	Near-real-time (<1 min)	Redis + feature store	Recent activity, session data
On-demand	Per request	Computed in serving layer	Features that depend on request context

11.10.3 LLM-Specific Evaluation Frameworks

Offline Evaluation with Golden Datasets

```
# Curated test set with ground truth
eval_dataset = [
    {
        "input": "What is our return policy?",
        "expected": "You can return items within 30 days with receipt.",
        "context": [policy_document_chunk],
        "category": "policy_lookup"
    },
    # ... hundreds more
]

def evaluate_rag_system(dataset, rag_chain):
    results = []
    for example in dataset:
        response = rag_chain.invoke(example["input"])
        results.append({
            "faithfulness": faithfulness_score(response, example["context"]),
            "answer_relevance": relevance_score(response, example["input"]),
            "correctness": correctness_score(response, example["expected"]),
            "latency_ms": response.latency_ms
        })

    # Aggregate
    return {
        metric: sum(r[metric] for r in results) / len(results)
```

```

    for metric in ["faithfulness", "answer_relevance", "correctness"]
    }

```

RAGAS Metrics Explained

Metric	What It Measures	Good Score	Formula Intuition
Context Precision	Are retrieved chunks relevant?	>0.80	% of retrieved chunks that are actually useful
Context Recall	Are all relevant chunks retrieved?	>0.75	% of needed facts that appear in retrieved chunks
Answer Faithfulness	Does answer stay within retrieved context?	>0.85	% of answer claims that can be grounded in context
Answer Relevance	Does answer address the question?	>0.80	Semantic similarity of answer to question

LLM-as-Judge Evaluation

```

def llm_judge_faithfulness(response: str, context: str) -> float:
    judge_prompt = f"""You are evaluating an AI assistant's answer.

Context provided to the assistant:
{context}

Assistant's answer:
{response}

Task: List any claims in the answer that are NOT supported by the context.
If all claims are supported, say "ALL_SUPPORTED".

Output format: JSON {"unsupported_claims": [...], "verdict": "supported" |
"partial" | "hallucinated"}}
"""

    result = judge_llm.invoke(judge_prompt)
    verdict = parse_json(result)["verdict"]
    return {"supported": 1.0, "partial": 0.5, "hallucinated": 0.0}[verdict]

```

LLM-as-judge calibration tip: Always validate your judge model's scores against human labels on 200+ examples before trusting it in production. A judge model with systematic bias will give you consistent but wrong scores.

11.10.4 Hallucination Detection at Scale

Multi-Layer Detection Stack

```
Layer 1 – Fast heuristics (<5ms):
  Claim extraction: identify factual statements in response
  Token probability check: low-confidence tokens signal uncertainty
  Citation presence: did model cite a source it was given?

Layer 2 – Semantic verification (~20ms):
  For each claim: cosine_similarity(claim_embedding, context_chunks) > 0.8?
  NLI model: Does context entail the claim? (fast BERT-based NLI)

Layer 3 – LLM judge (~100ms, sampled 1-5%):
  Full faithfulness evaluation
  Used for: difficult cases, calibration, regression dataset building

Layer 4 – Human review (minutes, sampled 0.1%):
  Random sample + all flagged low-confidence responses
  Gold labels feed back into Layer 3 calibration
```

Production Monitoring

```
class HallucinationMonitor:
    ALERT_THRESHOLDS = {
        "faithfulness_warning": 0.80,
        "faithfulness_page": 0.65,
        "spike_detection": 0.10, # >10% drop vs rolling 24h avg
    }

    def check_response(self, response: str, context: str) -> MonitorResult:
        score = self.fast_check(response, context) # Layer 1+2

        self.metrics.record("faithfulness_score", score)

        if score < self.ALERT_THRESHOLDS["faithfulness_page"]:
            self.alert("HIGH_SEVERITY_HALLUCINATION", score, response)
        elif score < self.ALERT_THRESHOLDS["faithfulness_warning"]:
            self.queue_for_review(response, context, score)

        # Spike detection: rolling average
        rolling_avg = self.metrics.rolling_mean("faithfulness_score", window="1h")
        if rolling_avg < self.metrics.baseline -
self.ALERT_THRESHOLDS["spike_detection"]:
            self.alert("HALLUCINATION_RATE_SPIKE", rolling_avg)

        return MonitorResult(score=score, flagged=score <
self.ALERT_THRESHOLDS["faithfulness_warning"])
```

11.10.5 Prompt Versioning — LLM-Specific MLOps

Unlike traditional ML, LLMs are sensitive to prompt wording. Prompt changes are deployable artifacts that need the same lifecycle rigor as model artifacts.

Prompt Registry (extend model registry concept):

```
prompt_registry.register(  
    name="customer-support-system-prompt",  
    version="v3.2",  
    content="""You are a customer support agent for Acme Corp...""",  
    metadata={  
        "author": "ml-team",  
        "eval_score": 0.91,          # Evaluated before registration  
        "tested_on_models": ["claude-sonnet-4-6", "gpt-4o"],  
        "deployed_with_model": "claude-sonnet-4-6",  
    }  
)
```

Deployment rule: Model version + Prompt version are deployed together.
Changing either = new combined version, new A/B test required.

Interview Problems

Problem 1: RAG Evaluation System with Hallucination Detection at Scale

Scenario: Your RAG chatbot serves 100K queries/day. Design an evaluation system that detects hallucinations without evaluating every query (too expensive).

Solution:

```
class ProductionRAGEvaluator:  
    SAMPLE_RATES = {  
        "random_sample": 0.01, # 1% of all traffic  
        "low_confidence": 1.00, # 100% of flagged responses  
        "new_model_rollout": 0.10, # 10% during A/B tests  
        "high_stakes_category": 0.05, # 5% of medical/legal queries  
    }  
  
    def should_evaluate(self, request: RAGRequest, response: RAGResponse) -> bool:  
        if response.token_confidence < 0.7:  
            return True # Always evaluate low-confidence  
        if request.category in ["medical", "legal", "financial"]:  
            return random.random() < self.SAMPLE_RATES["high_stakes_category"]  
        return random.random() < self.SAMPLE_RATES["random_sample"]
```

```

def evaluate(self, request, response):
    scores = {
        "faithfulness": self.faithfulness_check(response.text,
response.context),
        "relevance": self.relevance_check(response.text, request.query),
        "context_quality": self.context_precision(response.context,
request.query)
    }

    # Feed regression test dataset
    if scores["faithfulness"] < 0.6:
        self.regression_dataset.add({
            "query": request.query,
            "bad_response": response.text,
            "context": response.context,
            "label": "hallucinated"
        })

    return scores

# Daily evaluation summary:
# - 1% of 100K = 1,000 random evaluations
# - ~50 low-confidence responses evaluated fully
# - Cost: 1,050 LLM judge calls × ~$0.002 = ~$2/day

```

Problem 2: Feature Store for a Churn Prediction Model

Scenario: You train a churn model daily on 3 months of user activity. At serving time, you need the same features instantly. The features include rolling aggregates (e.g., “sessions in last 7 days”). Design the feature pipeline.

Solution:

```

Feature definitions (Feast):

user_activity_features = FeatureView(
    name="user_activity",
    entities=["user_id"],
    ttl=timedelta(days=7), # Refresh at least weekly
    features=[
        Feature("sessions_7d", dtype=Int32), # Rolling 7-day session count
        Feature("avg_session_min", dtype=Float32), # Avg session duration
        Feature("days_since_login", dtype=Int32),
        Feature("pages_viewed_7d", dtype=Int32),
    ],
    source=batch_source # BigQuery table
)

Pipeline:
Daily batch job (2 AM):

```


- Query event table for last 7 days
- Compute rolling aggregates per user
- Write to feature store (Redis online + BigQuery offline)
- Validate: <1% null values, distributions within expected range

Serving (real-time):

GET /features/user_activity?user_id=123

→ Redis lookup: <5ms

→ Cache miss: BigQuery fallback: <200ms

Training (offline):

Historical fetch with point-in-time join

Ensures no data leakage (future features not used for past labels)

Problem 3: Detecting Model Quality Degradation in Production

Scenario: Your model's accuracy was 88% at launch. Six months later, users report worse answers but no one changed the model. How do you diagnose silent degradation?

Solution:

Root causes of silent degradation (no code change):

1. Data drift: Input distribution changed
Check: Compare embedding distribution of recent vs. launch queries
Tool: Wasserstein distance between query embedding distributions
Signal: Distance > 0.15 → meaningful drift
2. Concept drift: World knowledge became stale
Check: Queries about recent events return hallucinated answers
Signal: Spike in "I don't know" or factually incorrect responses
Fix: Re-ingestion pipeline, RAG document refresh
3. Feature drift: Feature store values shifted
Check: Compare feature distribution statistics (mean, std, percentiles)
Tool: Great Expectations data validation
Signal: Feature mean shifted >2 standard deviations
4. Upstream data change: Source DB schema change broke feature computation
Check: Feature pipeline monitoring, null rate per feature
Signal: Null rate for "sessions_7d" jumped from 0.1% to 15%

Monitoring dashboard (built at launch, not after the fact):

- ☐ Query embedding drift score (daily)
- ☐ Response faithfulness score (sampled, daily)
- ☐ Feature null rates (per feature, hourly)
- ☐ Feature distribution statistics (daily)
- ☐ User feedback score (thumbs up/down, daily)

Alerts:

- Any metric outside 2σ of baseline → investigation ticket
- Any metric outside 3σ → page on-call engineer